Homework 1 (Matlab version)

EK505 - Prof. Tron

2026-09-15

Goal The goals of this assignment are to set up the Python environment for the entire semester and practice the kinematic modeling and visualization of serial robots.

Environment setup

Preparation. For all the homework this semester, we will use an environment with the Mamba Python package manager and the Visual Studio Code editor. You may use a different setup, but this is the configuration that will be supported in the course. To prepare your environment, please use the following steps:

1) Install the Mamba package manager (Miniforge distribution) by following the instructions at `https://github.com/conda-forge/miniforge?tab=readme-ov-file` (scroll down to the Install section).
Warning: On macOS and Linux, you can install Mamba next to a pre-existing Conda/Anaconda installation. However, you need to disable the automatic activation of the base environment. On Windows, you can safely install Mamba next to a Conda/Anaconda installation, but make sure to use the Miniconda Prompt for the operations below.

🔎 It should be possible to follow most of the homework assignments using a Conda/Anaconda distribution. However, some packages might not be available or might have different versions. It is recommended that you use Mamba.

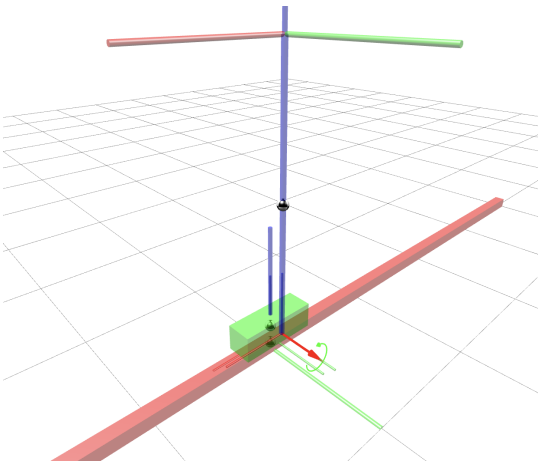
2) From a terminal prompt (Terminal on macOS, Miniconda Prompt on Windows) run the following commands:

(a) Use `cd` to navigate to the folder containing the provided file `ek505_environment.yml`. Use the command `mamba env create -y -n ek505 -f ek505_environment.yml`. This will create a `ek505` Mamba environment with all the required packages.
🔎 If you are not using the Mamba distribution: Most of the packages come from the `conda-forge` channel. However, the `roboticstoolbox-python` package is installed via `pip` to access the latest version.

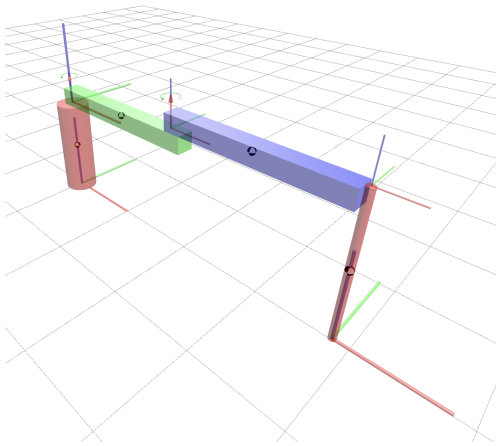
(b) Activate the environment using `mamba activate ek505`.

(c) Use the command below to verify that the installation of the Robotics Toolbox is complete.

```
python -c "import roboticstoolbox as rtb; rtb.models.catalog()"
```



(a) Cart-pole robot



(b) SCARA robot

Figure 1: Visualization of the two robots for this assignment, with centers of mass and axes represented according to the conventions of the viewer `viewer.robotsfan.com`.

It might take a couple of minutes to execute (during which the terminal might seem non-responsive). After it finishes, you should see a table of robots available in the Robotics Toolbox.

(d) You may close the terminal.

3) Download and install Visual Studio Code from `https://code.visualstudio.com/`. Start the application, and install the following extensions, if they are not already present (click the icon with four squares in the left toolbar and then search for the extensions): Python ,Pylance ,Python Debugger ,Python Environments ,Black Formatter ,Jupyter ,Container Tools ,Dev Containers . Optionally, also install the vscode-pdf extension to be able to see the PDFs of the assignments side by side with your code.

Problem 1: Robot kinematic models

Question report **1.1.** quiz In regard to URDF files, explain:

- What is the purpose of the format, and what are its limitations?
- Where does the syntax come from?
- What are the main elements that define a robot model?
- How do the elements relate to each other?

Question code **1.1.** Prepare a URDF file `cartpole.urdf` for a cart-pole system (Figure 1a). The system should have four links (`base_link` , `cart` , `pole` , `ee_link`), one prismatic joint (`base_to_cart`), one continuous joint (`cart_to_pole`), and one fixed joint (`pole_to_ee`). The `base_link` should be about 4 m along the x axis, be much heavier than the rest of the robot, and sit on the ground plane. The `cart` link should slide on top of the `base_link` and have mass of 1 kg; the `base_to_cart` joint should have limits so that the cart does not leave the base. The `pole` link should be about 1 m long and have mass that is

at least $\frac{1}{10}$ of the mass of the `cart`. The `cart_to_pole` joint should be along the y axis and should not have joint limits. Use the figure as a guide for the rest of the details (such as colors); exact dimensions are not important, as long as the proportions are approximately correct and the inertia matrices are consistent. For the computation of the inertia matrices, you may use Blender with the LinkForge extension or the table provided on Blackboard.

Question code 1.2. Repeat Question code 1.1 for a SCARA robot in the file `scara.urdf` (Figure 1b). The system should have five links (`base_link`, `link1`, `link2`, `plunger`, `ee_link`), two revolute joints (`base_to_link1`, `link1_to_link2`), one prismatic joint (`link2_to_plunger`, along the z axis), and one fixed joint (`plunger_to_ee`). The `base_link` should be cylindrical, much heavier than the rest, and sit on the ground plane. Each one of `link1` and `link2` should be 1 m along the x axis. The `base_to_link1` joint should have limits close to the $[-\pi, \pi]$ range, while the limits for `link1_to_link2` should prevent self-collisions of the plunger with the rest of the assembly. The end effector should be capable of touching the ground plane at maximum extension.

Question Pre-LLM 1.1. Given the images in Question 1, draw sketches of the robots that include dimensions, inertia matrices, placement of the links' origin, and placement of the joints (with axes). Then, write a draft of the URDF file contents.

Question optional 1.1. Write and submit on Gradescope a Python script that computes inertia matrices, to preplace the table provided on Blackboard.

Question report 1.2. Include plots of the robots in different positions using the Swift simulator or the visualization tool available at `viewer.robotsfan.com`. For each position, include three approximately orthographic views showing the robot from the top, front, and right side. In the report, the three views should appear in the same row or the same column for ease of understanding.

Question code 1.3. In this question, we first create a simple joint interpolation function, and then use it to animate the robot in the Swift simulator.

> File name: `ek505_utilities.py`
>
> Function name: `interp_pingpong`
>
> Description: Generate a "ping-pong" trajectory from a sequence of robot configuration waypoints. The function first linearly interpolates between each pair of consecutive waypoints using `nb_per_segment` points per segment. It then appends the reversed trajectory to the end, so that the resulting trajectory moves forward through the waypoints and then backward through the same path.
>
> Input arguments
>
> - `q_waypoints` (length `nb_waypoints`, type `list`): list of lists (each one of length `nb_joints`) with joint angles defining the waypoints of the trajectory.
> - `nb_per_segment`: number of interpolation points used between each pair of consecutive waypoints. The default value is `100`.
>
> Output arguments
>
> - `q_traj` (dim. [`2(nb_waypoints-1)(nb_per_segment-1)` × `nb_joints`], type

`np.array`): array containing the interpolated ping-pong trajectory. Each row is one configuration along the trajectory.

🔎 The trajectory should not contain repeated points (both between segments, and between the beginning and the end). For instance, the following call with 3 waypoints and 3 points per segment

`interp_pingpong([[0,0,1],[0,1,0],[1,0,0]],nb_per_segment=3)`

should produce the following trajectory with $2 * (3-1) * (3-1) = 8$ configurations.

```
array([[0.  , 0.  , 1. ],
       [0.  , 0.5, 0.5],
       [0.  , 1.  , 0. ],
       [0.5, 0.5, 0. ],
       [1.  , 0.  , 0. ],
       [0.5, 0.5, 0. ],
       [0.  , 1.  , 0. ],
       [0.  , 0.5, 0.5]])
```

Next, create the function that performs the animation.

File name: `ek505_hw1.py`
Function name: `animation_interpolation_test`
Description: Define and animate a mock pick-and-place sequence for the SCARA robot. The function specifies a sequence of joint-space waypoints, generates a "ping-pong" trajectory through these waypoints using `interp_pingpong`, and then visualizes the resulting motion in the Swift simulator. The animation repeatedly moves the robot forward and backward along the interpolated trajectory.

Question video 1.1. Submit a screen recording of the robot animation.

Question Pre-LLM 1.2. Explain how to use the contents of a URDF file to write the forward kinematics of a robot as the product of 4×4 homogeneous transformations. Then, explain how to obtain each element using the classes and methods available in the `spatialmath` package.

Question report 1.3. Write an expression for computing the pose of the end of the pole in the cart-pole robot (i.e., the `ee_link`) as the product of 4×4 transformations. To obtain full marks, include an explanation of how each factor was derived.

Question provided 1.1. The file `ek505_models.py` defines two module-level variables `scara` and `cartpole` (both of type `roboticstoolbox.models.URDF.URDFRobot.URDFRobot`) which are obtained by loading, respectively, the `scara.urdf` and `cartpole.urdf` files from the current directory.

Question code 1.4. Write a function implementing Question report 1.3.

File name: `ek505_models.py`
Function name: `cartpole_fkine`
Description: Compute the forward kinematics of the cart-pole robot by explicitly con-

catenating homogeneous transformations.
Input arguments

- `q` (length `nb_joints`, type `list`): configuration of the cart-pole robot.

Output arguments

- `T` (type `SE3`): pose of the end of the pole, represented as an element of `spatialmath.SE3`.

🔎 You may use only function from the `spatialmath` module, not from the `roboticstoolbox` module.

Question report 1.4. quiz Repeat Question report 1.3, but for the SCARA robot.

Question code 1.5. Write a function implementing Question report 1.4.

File name: `ek505_models.py`
Function name: `scara_fkine`
Description: Compute the forward kinematics of the SCARA robot by explicitly concatenating homogeneous transformations.
Input arguments

- `q` (length `nb_joints`, type `list`): configuration of the SCARA robot.

Output arguments

- `T` (type `SE3`): end-effector pose of the SCARA robot, represented as an element of `spatialmath.SE3`.

Question provided 1.2. The following functions show, next to each other, the results of the forward kinematics computed with Robotics Toolbox and with the hand-written pose composition for the cart-pole and SCARA robot.

File name: `ek505_hw1.py`
Function name: `cartpole_fkine_test`
Description: Show the results of the forward kinematics computed for the cart-pole robot using both the Robotics Toolbox model and the hand-written pose composition implemented in `cartpole_fkine` for several randomly generated configurations.

File name: `ek505_hw1.py`
Function name: `scara_fkine_test`
Description: Similar to `cartpole_fkine_test()`, but for the SCARA robot.

Question report 1.5. Include the code written for Questions code 1.4, code 1.5, and the outputs of Question provided 1.2 in your report.